**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**

**KHOA CÔNG NGHỆ THÔNG TIN**

BỘ MÔN HỆ ĐIỀU HÀNH




# BÁO CÁO THỰC HÀNH

## MÔN HỆ ĐIỀU HÀNH

# Lab 01

# Xv6 User Programs: File System

**Lớp học phần:** Hệ điều hành - CQ2024/22

**Sinh viên:** 24120438 – Trần Nguyên Tân

24120023 – Nguyễn Thành Bảo

24120040 – Dương Hoài Đức

**Giảng viên:** ThS. Lê Viết Long

TP. HỒ CHÍ MINH, THÁNG 10/2026

# Mục lục

**Tóm tắt báo cáo** **3**

**1 Thông tin thành viên và phân công công việc** **4**

**2 Thiết lập môi trường trên Windows và macOS** **4**

2.1 Windows: Ubuntu và dependencies trên WSL 2 . . . . . 4

2.2 macOS: cài đặt công cụ RISC-V qua Homebrew . . . . . 5

2.3 Chuẩn bị mã nguồn và khởi chạy xv6 . . . . . 6

2.4 Môi trường dùng để ghi nhận kết quả . . . . . 6

**3 Kiến thức nền tảng** **7**

3.1 Mô hình thực thi chương trình người dùng . . . . . 7

3.2 Các system call liên quan . . . . . 9

3.3 Các hàm thư viện người dùng . . . . . 12

3.4 Tiến trình và cơ chế giao tiếp liên tiến trình qua đường ống . . . . 15

3.5 Hệ thống tập tin và kỹ thuật duyệt cây thư mục . . . . . 16

3.6 Xử lý dòng dữ liệu I/O và nguyên tắc quản trị tài nguyên an toàn . 19

**4 Các chương trình đã thực hiện** **20**

4.1 sleep . . . . . 20

4.2 pingpong . . . . . 22

4.3 primes . . . . . 24

4.4 cp . . . . . 26

4.5 tree . . . . . 29

4.6 du . . . . . 31

4.7 diff . . . . . 33

**5 Kết quả kiểm thử tổng hợp** **37**

5.1 Môi trường và cách thực hiện . . . . . 37

5.2 Kết quả theo từng chương trình . . . . . 38
5.3 Nhật ký thực thi và phạm vi kết luận . . . . . 39

**6 Khó khăn gặp phải và giải pháp khắc phục 39**

**7 Kết luận và hướng phát triển 40**

**Tài liệu tham khảo 40**

# Tóm tắt báo cáo

Báo cáo này trình bày kết quả thực hiện, thiết kế kỹ thuật và quá trình kiểm thử toàn diện các chương trình tiện ích không gian người dùng (*User Programs*) trong khuôn khổ bài thực hành **Lab 01 (Hệ điều hành xv6 – Kiến trúc RISC-V)**. Mục tiêu trọng tâm của bài thực hành là tiếp cận kiến trúc phân tầng người dùng/kernel (*User/Kernel Space*), vận dụng các system call cơ bản, cơ chế quản lý vòng đời tiến trình, truyền thông liên tiến trình (IPC) và các thao tác truy xuất hệ thống tệp tin (*File System*).

Trong suốt quá trình thực hiện, nhóm đã phân tích yêu cầu, xây dựng mã giả và hiện thực thành công toàn bộ **07 chương trình tiện ích**:

- sleep: Tạm dừng tiến trình theo chu kỳ đồng hồ (*ticks*) của hệ thống.
- pingpong: Hiện thực cơ chế giao tiếp 2 chiều đồng bộ giữa tiến trình cha và con thông qua cặp đường ống (*pipes*).
- primes: Xây dựng thuật toán sàng số nguyên tố Eratosthenes đồng thời (*Concurrent Prime Sieve*) theo mô hình pipeline đa tiến trình kết nối liên tiếp từ 2 đến 280.
- cp: Sao chép dữ liệu tập tin an toàn qua bộ đệm (*buffer streaming*), xử lý kiểm tra trùng tệp nguồn/đích.
- tree: Duyệt đệ quy cây thư mục hiển thị phân cấp trực quan, hỗ trợ giới hạn độ sâu (-L) và lọc thư mục (-d).
- du: Ước tính dung lượng sử dụng đĩa (*Disk Usage*) theo byte, hỗ trợ tính tổng nhanh (-s) và liệt kê chi tiết tệp (-a).
- diff: So sánh nội dung hai tệp tin theo từng dòng, xử lý so khớp EOF và chế độ thông báo tóm tắt (-q).

**Tóm tắt kết quả & Đánh giá chung:**

- **Tỷ lệ hoàn thành:** 7/7 chương trình (100% khối lượng yêu cầu).
- **Độ tin cậy & Quản lý tài nguyên:** Tất cả các chương trình đều được kiểm thử nghiêm ngặt trên trình giả lập QEMU, kiểm soát chặt chẽ việc giải phóng bộ mô tả tập tin (không rò rỉ File Descriptor), đóng đúng các đầu pipe thừa để kích hoạt tín hiệu EOF chính xác, và đồng bộ hóa triệt để bằng wait() nhằm loại bỏ hoàn toàn nguy cơ sinh tiến trình ma (*zombie process*).
- **Kiểm thử tự động:** Đáp ứng đầy đủ các trường hợp kiểm thử chức

> năng và edge cases lỗi theo chuẩn kịch bản kiểm thử của môn học.

# 1. Thông tin thành viên và phân công công việc

| STT | MSSV | Họ và tên | Nhiệm vụ phụ trách |
|---|---|---|---|
| 1 | 24120438 | Trần Nguyên Tân | sleep, pingpong, primes |
| 2 | 24120023 | Nguyễn Thành Bảo | cp, diff |
| 3 | 24120040 | Dương Hoài Đức | tree, du, tích hợp và kiểm thử chung |

# 2. Thiết lập môi trường trên Windows và macOS

Do xv6 trong bài thực hành chạy trên kiến trúc RISC-V. Máy chủ đảm nhiệm việc biên dịch mã nguồn, còn QEMU mô phỏng phần cứng để khởi động xv6. Vì vậy, dù dùng Windows hay macOS, môi trường cần có trình biên dịch chéo RISC-V, bộ công cụ liên kết và QEMU; trình biên dịch C mặc định của máy không thay thế được bộ công cụ RISC-V.

## 2.1 Windows: Ubuntu và dependencies trên WSL 2

Trên Windows, quá trình biên dịch và kiểm thử được thực hiện trong Ubuntu trên WSL 2. Cách tổ chức này giữ được môi trường Linux theo yêu cầu bài thực hành mà vẫn cho phép soạn thảo tài liệu và quản lý mã nguồn từ Windows. Từ hướng dẫn của MIT, nhóm xác định sử dụng WSL 2 và QEMU từ phiên bản 7.2; Ubuntu phiên bản 24.04 để thuận tiện cài các gói phù hợp [6].

**Cài đặt WSL.** Mở PowerShell với quyền Administrator, cài Ubuntu rồi khởi động lại máy nếu được yêu cầu. Khi mở Ubuntu lần đầu, tạo tài khoản và mật khẩu Linux. Các lệnh dưới đây áp dụng cho Windows 10 phiên bản 2004 trở lên hoặc Windows 11 [9].

```
wsl --install -d Ubuntu-24.04
wsl --list --verbose
```

**Cài công cụ phát triển.** GCC và Binutils cho RISC-V dùng để tạo chương trình chạy trên xv6; build-essential cung cấp công cụ biên dịch phía máy chủ, QEMU

chạy máy mô phỏng, còn Python dùng cho bộ kiểm thử của dự án. gdb-multiarch hỗ trợ gỡ lỗi khi cần.

```
sudo apt update
sudo apt install build-essential git perl python3 \
  gdb-multiarch qemu-system-misc \
  gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu

riscv64-linux-gnu-gcc --version
qemu-system-riscv64 --version
python3 --version
```

## 2.2 macOS: cài đặt công cụ RISC-V qua Homebrew

Trên macOS, có thể biên dịch trực tiếp trong Terminal và chạy xv6 bằng QEMU. Trước hết, cài Command Line Tools để có công cụ phát triển phía máy chủ, sau đó cài Homebrew theo hướng dẫn chính thức [10].

```
xcode-select --install
```

Sau khi cài Homebrew, thực hiện phần cấu hình brew shellenv được trình cài đặt hướng dẫn rồi mở lại Terminal. Đường dẫn này phụ thuộc vào cách cài và kiến trúc máy; không nên sao chép một đường dẫn cố định từ máy khác.

```
brew --version
brew install riscv64-elf-gcc qemu python

riscv64-elf-gcc --version
riscv64-elf-objdump --version
qemu-system-riscv64 --version
python3 --version
```

Gói riscv64-elf-gcc của Homebrew kéo theo Binutils tương ứng [11]. Đây là cách cài được dùng trong hướng dẫn này, thay cho cấu hình tap RISC-V trong tài liệu cũ. Khả năng cung cấp gói dựng sẵn phụ thuộc phiên bản macOS và kiến trúc máy, nên cần kiểm tra điều kiện hỗ trợ của Homebrew trước khi cài.

Bộ công cụ này có tiền tố riscv64-elf-, trong khi Makefile của mã nguồn hiện tại chưa tự nhận diện tiền tố đó. Vì vậy, khi biên dịch trên macOS phải truyền rõ TOOLPREFIX, thay vì đổi tên các chương trình hoặc sửa cố định đường dẫn trong Makefile.

## 2.3 Chuẩn bị mã nguồn và khởi chạy xv6

Nhóm đã sử dụng mã nguồn trong folder xv6-labs-2024/ của dự án, source code của xv6 được clone từ git://g.csail.mit.edu/xv6-labs-2024

Từ thư mục gốc dự án, chạy các lệnh sau để biên dịch:

Ubuntu trên WSL 2

```
cd xv6-labs-2024
make
make qemu
```

macOS

```
cd xv6-labs-2024
make TOOLPREFIX=riscv64-elf-
make TOOLPREFIX=riscv64-elf- qemu
```

Môi trường sẵn sàng khi xv6 khởi động sẽ xuất hiện init: starting sh và dấu nhắc shell. Khi đó ta có thể chạy các lệnh sleep 10, pingpong hoặc primes để kiểm tra bước đầu. Để thoát QEMU, nhấn Ctrl+A, thả phím rồi nhấn X.

Các chương trình người dùng được khai báo trong UPROGS của Makefile. Ví dụ, tệp dựng user/_sleep được đưa vào fs.img với tên sleep; khi chạy trong shell xv6 không thêm dấu gạch dưới. Các tệp tạo trong xv6 thuộc ảnh đĩa này, không xuất hiện như tệp thông thường trong thư mục máy chủ. Lệnh make clean xóa sản phẩm biên dịch và ảnh đĩa, nên cần lưu dữ liệu thử nghiệm quan trọng trước khi dọn bản dựng.

**Kiểm thử tự động.** Từ thư mục gốc dự án, dùng python3 tests/lab01/run.py trên WSL. Trên macOS, thêm tùy chọn chọn bộ công cụ:

```
python3 tests/lab01/run.py --toolprefix=riscv64-elf-
```

## 2.4 Môi trường dùng để ghi nhận kết quả

Các thao tác kiểm thử được báo cáo ở mục 5 được thực hiện trong Ubuntu 24.04 qua WSL, với môi trường cài đặt GCC RISC-V 13.3.0 và QEMU 8.2.2. Đây là môi trường đã tạo ra kết quả 77/77 ca kiểm thử của báo cáo.

# 3. Kiến thức nền tảng

Để xây dựng các chương trình tiện ích không gian người dùng (*user programs*) trong bài thực hành Lab 01, nhóm đã tìm hiểu và vận dụng các kiến thức cốt lõi về giao diện hệ điều hành và hệ thống tập tin được trình bày trong tài liệu *xv6 book* (đặc biệt là Chương 1: *Operating system interfaces* và Chương 8: *File system*) [1].

Mục này trình bày có hệ thống các cơ chế nền tảng của xv6 liên quan trực tiếp đến bài thực hành, bao gồm: mô hình phân tầng không gian người dùng và kernel, hệ thống lời gọi hệ thống (system call) cùng thư viện người dùng, cơ chế quản lý vòng đời tiến trình và giao tiếp qua đường ống (pipe), cấu trúc tổ chức của hệ thống tập tin (inode, directory entry), cũng như các kỹ thuật xử lý I/O và nguyên tắc quản trị tài nguyên an toàn.

## 3.1 Mô hình thực thi chương trình người dùng

Hệ điều hành xv6 vận hành trên kiến trúc vi xử lý RISC-V, tận dụng cơ chế phân quyền phần cứng để chia tách thành hai chế độ thực thi: *chế độ người dùng* (User mode) và *chế độ giám sát* (Supervisor mode). Tất cả các tiện ích người dùng (`sleep`, `pingpong`, `cp`,...) đều được nạp và thực thi trong **không gian người dùng** (*user space*). Tại đây, mỗi tiến trình sở hữu một không gian địa chỉ ảo riêng biệt được ánh xạ thông qua bảng trang (*page table*). Cơ chế phân lập này giúp bảo vệ tính toàn vẹn của hệ thống: một chương trình người dùng không thể can thiệp hay đọc trộm vùng nhớ của tiến trình khác, đồng thời không thể truy cập trực tiếp vào các cấu trúc dữ liệu nội bộ của kernel.

Khi cần thao tác với phần cứng hoặc tài nguyên dùng chung—chẳng hạn như đọc ghi dữ liệu trên đĩa, tạm dừng theo nhịp đồng hồ, hay truyền dữ liệu cho tiến trình khác—chương trình phải phát ra một **lời gọi hệ thống** (*system call*). Trên nền tảng RISC-V, thao tác này được kích hoạt thông qua chỉ thị `ecall`, tạo ra một bẫy phần mềm (*trap*) chuyển quyền kiểm soát sang **không gian kernel** (*kernel space*). Kernel sẽ xác thực tham số, thực thi tác vụ tương ứng với đầy đủ đặc quyền phần cứng rồi trả kết quả về cho tiến trình người dùng tiếp tục luồng thực thi.

**Cơ chế nạp lệnh của Shell: Sự phối hợp giữa fork() và exec()**

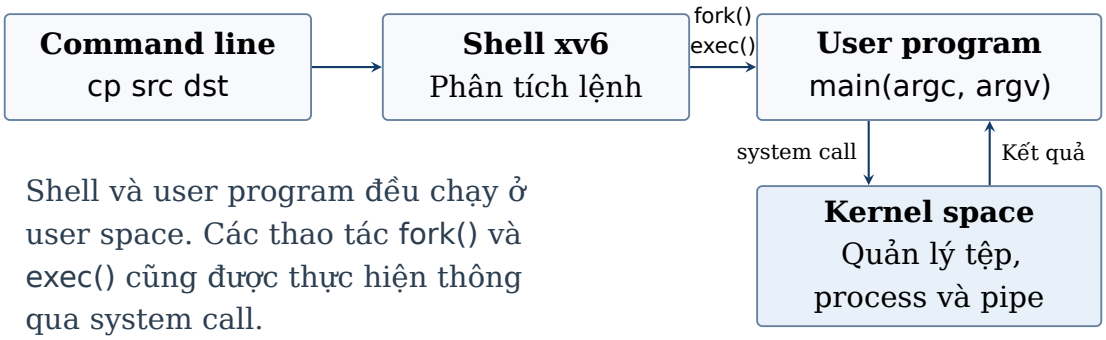
Điểm độc đáo trong thiết kế của Unix được xv6 kế thừa trọn vẹn là sự tách biệt rõ ràng giữa hai tác vụ: tạo tiến trình mới và nạp mã chương trình thực thi. Khi người dùng nhập một câu lệnh vào dấu nhắc shell (chẳng hạn `cp src dst`):

1. Shell (vốn cũng là một tiến trình chạy ở user space) đọc và phân tích cú pháp chuỗi dòng lệnh thành tên chương trình (`"cp"`) cùng mảng các đối số

(argv).

2. Shell gọi system call fork() để tạo một tiến trình con (*child process*). Tiến trình con này ban đầu là bản sao chính xác về không gian nhớ và trạng thái các tập tin mô tả đang mở của tiến trình cha.
3. Trong tiến trình con, shell gọi tiếp system call exec() để thay thế toàn bộ ảnh bộ nhớ hiện tại bằng mã máy nhị phân của chương trình cp được nạp từ hệ thống tập tin, sau đó bắt đầu thực thi từ hàm main().
4. Trong khi tiến trình con thực hiện nhiệm vụ sao chép, tiến trình shell cha gọi wait() để tạm dừng cho đến khi tiến trình con hoàn tất và trả về mã trạng thái thoát, rồi mới hiển thị lại dấu nhắc lệnh.

Sự tách biệt giữa fork() và exec() mang lại tính linh hoạt rất lớn: nó cho phép shell can thiệp chuyển hướng I/O (*I/O redirection*) hoặc thiết lập đường ống (*pipe*) ngay trong tiến trình con *trước khi* mã chương trình mới thực sự được nạp, mà không làm xáo trộn cấu hình của tiến trình cha hay đòi hỏi kernel phải can thiệp vào logic của từng lệnh.




*Luồng khởi chạy một lệnh và tương tác giữa user program với kernel.*

### Xử lý tham số dòng lệnh (argc, argv)

Khi nạp một chương trình qua exec(), kernel thiết lập sẵn trên đỉnh ngăn xếp (stack) của tiến trình mới hai tham số: số lượng đối số argc và mảng con trỏ chuỗi ký tự argv. Theo chuẩn ngôn ngữ C trong xv6:

- argv[0] luôn là chuỗi chứa tên lệnh được gọi (chẳng hạn "sleep", "cp").
- Các phần tử tiếp theo từ argv[1] đến argv[argc-1] chứa các đối số dòng lệnh mà người dùng truyền vào.
- Phần tử kết thúc mảng argv[argc] luôn là một con trỏ NULL.

Vì toàn bộ đối số ban đầu đều ở dạng chuỗi ký tự thô, các chương trình cần tham số định lượng (như số nhịp chờ của sleep) hoặc các cờ tùy chọn (như -L trong tree, -s trong du, -q trong diff) bắt buộc phải tự kiểm tra tính hợp lệ về

cú pháp, phòng ngừa trường hợp thiếu đối số, truyền ký tự không hợp lệ hoặc vượt ngưỡng cho phép trước khi thực thi nghiệp vụ chính.

## 3.2 Các system call liên quan

System call là giao diện lập trình duy nhất kết nối giữa ứng dụng người dùng và kernel. Trong xv6, mã nguồn kernel định nghĩa danh sách system call trong kernel/syscall.h và cài đặt hàm xử lý tại kernel/sysproc.c và kernel/sysfile.c.

**Tập tin mô tả (File Descriptors) và tính trừu tượng I/O**

Theo triết lý Unix, xv6 trừu tượng hóa hầu hết các đối tượng I/O—từ tệp tin thông thường, thư mục, thiết bị hiển thị console cho đến đường ống liên tiến trình—dưới dạng một chuỗi byte truy xuất thông qua một số nguyên nhỏ gọi là **tập tin mô tả** (*file descriptor - fd*).

Mỗi tiến trình duy trì một bảng các tập tin đang mở (proc->ofile[NOFILE]). Giá trị fd mà tiến trình nhận được thực chất là một chỉ mục (*index*) trỏ vào bảng này. Theo quy ước mặc định:

- fd = 0: Luồng nhập chuẩn (*Standard Input - stdin*), nhận dữ liệu từ bàn phím.
- fd = 1: Luồng xuất chuẩn (*Standard Output - stdout*), in dữ liệu ra màn hình.
- fd = 2: Luồng báo lỗi chuẩn (*Standard Error - stderr*), in thông báo lỗi.

Một nguyên tắc cốt lõi trong xv6 là kernel luôn cấp phát **chỉ số descriptor nhỏ nhất chưa được sử dụng** (*lowest-numbered unused descriptor*). Đặc tính này cho phép shell dễ dàng thực hiện chuyển hướng I/O: chỉ cần gọi close(0) rồi gọi open("input.txt", O_RDONLY), tệp vừa mở sẽ tự động chiếm lấy fd = 0.

Dưới đây là bảng tổng hợp các system call được sử dụng trực tiếp trong bài thực hành, phân chia theo nhóm chức năng:

**Nhóm quản lý tiến trình, thời gian chờ và IPC:**

| System call | Chức năng và tham số | Kết quả trả về và lưu ý kỹ thuật |
|---|---|---|
| fork() | Tạo một tiến trình con mới nhân bản từ tiến trình hiện tại. | Trả về $0$ trong tiến trình con; trả về PID của con trong tiến trình cha; trả về $-1$ nếu thất bại. Cả hai tiến trình cùng tiếp tục chạy độc lập từ sau vị trí gọi fork(). |
| wait(status) | Tạm dừng tiến trình cha cho đến khi một tiến trình con kết thúc; thu hồi tài nguyên và mã thoát. | Trả về PID của tiến trình con vừa kết thúc, hoặc $-1$ nếu không còn con để chờ. Nếu con trỏ status khác NULL, kernel sẽ ghi mã thoát của con vào địa chỉ này. Dùng wait(0) khi không cần nhận mã thoát. |
| exit(status) | Chấm dứt tiến trình gọi với mã trạng thái status. | Không bao giờ quay lại nơi gọi. Kernel tự động đóng các file descriptor còn mở của tiến trình và chuyển sang trạng thái zombie để chờ cha thu hồi. |
| getpid() | Lấy mã định danh (PID) của tiến trình đang thực thi. | Trả về số nguyên dương PID; dùng để nhận diện và phân biệt giữa tiến trình cha và con trong kết quả hiển thị (như trong pingpong). |
| sleep(ticks) | Tạm dừng tiến trình trong một số lượng chu kỳ ngắt định thời. | Trả về $0$ khi hoàn tất thời gian chờ, $-1$ nếu bị hủy. Đơn vị ticks tính theo xung nhịp ngắt của phần cứng, không tương đương với giây thời gian thực. |
| pipe(p) | Tạo kênh truyền dữ liệu một chiều; ghi hai descriptor vào mảng p. | Trả về $0$ khi thành công, $-1$ nếu xảy ra lỗi. p[0] là đầu đọc (*read end*), p[1] là đầu ghi (*write end*). |

**Nhóm thao tác tập tin, I/O và nạp chương trình:**

| System call | Chức năng và tham số | Kết quả trả về và lưu ý kỹ thuật |
|---|---|---|
| open(path, mode) | Mở hoặc tạo một tập tin theo đường dẫn với chế độ truy cập xác định. | Trả về file descriptor không âm nhỏ nhất còn trống nếu thành công; trả về $-1$ nếu thất bại. Các cờ chế độ (O_RDONLY, O_CREATE,...) được định nghĩa trong kernel/fcntl.h. |
| close(fd) | Đóng tập tin mô tả fd và giải phóng vị trí tương ứng trong bảng quản lý của tiến trình. | Trả về $0$ khi thành công, $-1$ nếu descriptor không hợp lệ. Đối với pipe, đóng đầu ghi là điều kiện tiên quyết để bên đọc nhận biết tín hiệu kết thúc dữ liệu (EOF). |
| read(fd, buf, n) | Đọc tối đa n byte từ tệp, thiết bị hoặc pipe vào vùng đệm buf. | Trả về số byte thực tế đọc được; trả về $0$ khi chạm điểm kết thúc dữ liệu (EOF); trả về $-1$ khi gặp lỗi I/O. Dữ liệu đọc vào không tự động kèm ký tự kết thúc chuỗi NULL. |
| write(fd, buf, n) | Ghi n byte từ vùng đệm buf ra tập tin mô tả fd. | Trả về số byte thực tế đã ghi, hoặc $-1$ nếu xảy ra lỗi. Cần kiểm tra số byte thực tế để đảm bảo toàn bộ nội dung đã được ghi đầy đủ vào thiết bị lưu trữ. |
| fstat(fd, st) | Lấy thông tin siêu dữ liệu của tập tin đang mở vào cấu trúc struct stat. | Trả về $0$ khi thành công, $-1$ khi thất bại. Kết quả trả về gồm loại tệp (type), số hiệu thiết bị (dev), số Inode (ino) và kích thước tệp theo byte (size). |
| exec(path, argv) | Thay thế tiến trình hiện tại bằng chương trình thực thi mới tại đường dẫn path. | Giữ nguyên PID và các file descriptor đang mở. Nếu thành công, lời gọi không bao giờ quay lại nơi gọi; trả về $-1$ nếu không tìm thấy tệp hoặc lỗi định dạng nhị phân. |

## 3.3 Các hàm thư viện người dùng

Bên cạnh các system call, xv6 cung cấp một thư viện người dùng tối giản được khai báo trong user/user.h (triển khai tại user/ulib.c, user/printf.c, user/umalloc.c). Cần phân biệt rõ hai khái niệm này: system call yêu cầu sự can thiệp của kernel và tốn chi phí chuyển ngữ cảnh (trap context switch), trong khi các hàm thư viện người dùng chạy hoàn toàn trong user space và được liên kết tĩnh trực tiếp vào mã nhị phân của chương trình.

Một số hàm thư viện đóng vai trò là hàm bọc tiện ích (*wrapper*). Điển hình như stat(path, st): trong kernel xv6 không có system call nhận trực tiếp chuỗi đường dẫn để lấy thông tin tệp, mà chỉ có fstat(fd, st) thao tác trên file descriptor. Hàm thư viện stat() được xây dựng bằng cách gọi liên tiếp open(), fstat() và tự động gọi close() trước khi trả về.

Do thư viện của xv6 không đầy đủ như chuẩn GNU libc hay POSIX, các tiện ích phức tạp trong bài (chẳng hạn như đọc từng dòng trong diff hay duyệt phân cấp trong tree) đòi hỏi người lập trình phải tự xây dựng giải pháp xử lý dữ liệu và quản lý bộ nhớ dựa trên các hàm cơ bản sau:

| Hàm thư viện | Công dụng | Điểm cần lưu ý khi sử dụng |
|---|---|---|
| **Thao tác đường dẫn, chuỗi ký tự và dữ liệu** | | |
| stat(path, st) | Lấy thông tin siêu dữ liệu của tệp theo chuỗi đường dẫn. | Là hàm bọc gọi liên tiếp open(), fstat() và close(). Trả về $0$ khi thành công, $-1$ khi lỗi. Tự động đóng descriptor trước khi thoát. |
| atoi(s) | Chuyển đổi chuỗi ký tự chữ số thành số nguyên kiểu int. | Không cung cấp cơ chế báo lỗi khi chuỗi chứa ký tự phi số; chương trình cần chủ động kiểm tra tính hợp lệ của chuỗi đầu vào trước khi gọi hàm. |
| strcmp(a, b) | So sánh hai chuỗi ký tự kết thúc bằng ký tự NULL. | Trả về $0$ khi hai chuỗi giống hệt nhau; dùng để nhận diện cờ lệnh (-L, -d, -s,...) và phát hiện các thư mục đặc biệt (".", "..."). |
| strlen(s) | Tính độ dài chuỗi ký tự (không bao gồm ký tự NULL). | Chỉ áp dụng an toàn cho các chuỗi có ký tự kết thúc chuẩn; không dùng để suy ra kích thước của khối dữ liệu nhị phân hoặc vùng đệm chưa gán '\0'. |
| memcmp(a, b, n) | So sánh chính xác n byte giữa hai vùng nhớ. | Trả về $0$ nếu hai vùng nhớ giống nhau; đặc biệt phù hợp để so sánh các dòng văn bản có chứa byte NULL hoặc dữ liệu nhị phân trong diff. |

| Hàm thư viện | Công dụng | Điểm cần lưu ý khi sử dụng |
|---|---|---|
| **Cấp phát bộ nhớ và xuất dữ liệu định dạng** | | |
| memmove(dst, src, n) | Sao chép an toàn n byte từ vùng nhớ nguồn sang vùng nhớ đích. | Hỗ trợ trường hợp hai vùng nhớ bị chồng lấn (*overlapping*); cần đảm bảo vùng nhớ đích đã được cấp phát đủ dung lượng tối thiểu $n$ byte. |
| malloc(n), free(p) | Cấp phát và giải phóng bộ nhớ động trên vùng nhớ heap. | Bắt buộc kiểm tra con trỏ trả về từ malloc() để phòng ngừa lỗi cạn bộ nhớ; luôn giải phóng bộ nhớ bằng free() khi mở rộng hoặc tái sử dụng vùng đệm. |
| printf(fmt, ...), fprintf(fd, fmt, ...) | In chuỗi định dạng ra luồng xuất chuẩn (fd = 1) hoặc descriptor chỉ định. | Hỗ trợ các định dạng cơ bản: %d, %x, %p, %s, %c. Trong xv6, hai hàm này trả về kiểu void thay vì số ký tự như trong libc chuẩn. |

## 3.4 Tiến trình và cơ chế giao tiếp liên tiến trình qua đường ống

Để xây dựng các ứng dụng đa tiến trình như pingpong và primes, việc hiểu rõ cách thức hệ điều hành tạo lập, điều phối và truyền thông giữa các tiến trình là yếu tố quyết định.

**Quản lý tiến trình và đồng bộ hóa qua wait()**

Trong xv6, mỗi tiến trình là một thực thể độc lập được đại diện bởi cấu trúc struct proc trong kernel, bao gồm trạng thái thực thi (RUNNING, RUNNABLE, SLEEPING, ZOMBIE), định danh tiến trình (PID), không gian địa chỉ bảng trang, và con trỏ liên kết đến tiến trình cha. Lời gọi fork() sao chép toàn bộ không gian địa chỉ của cha sang một tiến trình con mới, đồng thời sao chép toàn bộ bảng các tập tin mô tả đang mở. Sau fork(), cả hai tiến trình tiếp tục chạy từ vị trí lệnh kế tiếp nhưng trong hai không gian nhớ riêng biệt.

Khi tiến trình con hoàn tất công việc, nó gọi exit(status). Lúc này, kernel thu hồi hầu hết tài nguyên bộ nhớ của con nhưng vẫn giữ lại cấu trúc proc của nó ở trạng thái ***ZOMBIE*** để lưu lại mã thoát. Tiến trình cha bắt buộc phải gọi wait() để thu nhận mã trạng thái này và cho phép kernel giải phóng hoàn toàn tiến trình con khỏi bảng tiến trình. Nếu tiến trình cha không gọi wait(), tiến trình con sẽ trở thành tiến trình ma (*zombie process*), gây rò rỉ ô nhớ trong bảng tiến trình hệ thống.

**Cơ chế hoạt động của Pipe và quy tắc EOF sống còn**

Đường ống (*pipe*) là kênh truyền dữ liệu một chiều giữa các tiến trình trong Unix. Trong nhân xv6, một pipe được hiện thực bằng một bộ đệm tròn (*circular buffer*) có kích thước cố định (512 byte). System call pipe(p) tạo ra đường ống và trả về hai tập tin mô tả: p[0] dùng để đọc dữ liệu và p[1] dùng để ghi dữ liệu. Dữ liệu ghi vào p[1] sẽ được lưu tạm trong bộ đệm của kernel cho đến khi tiến trình đọc lấy ra từ p[0] theo cơ chế FIFO.

Hành vi đồng bộ hóa của pipe tuân thủ các quy tắc nghiêm ngặt:

- Khi bộ đệm rỗng, thao tác read() trên đầu đọc sẽ bị tạm dừng (*block*), đưa tiến trình vào trạng thái ngủ để chờ dữ liệu được ghi vào.
- Khi bộ đệm đầy, thao tác write() trên đầu ghi cũng sẽ bị tạm dừng cho đến khi tiến trình đọc giải phóng bớt dung lượng trong bộ đệm.
- **Quy tắc nhận diện EOF (End-of-File):** Thao tác read() chỉ trả về 0 (tín hiệu kết thúc dữ liệu) khi và chỉ khi bộ đệm của pipe đã rỗng **VÀ toàn bộ các tập tin mô tả trỏ đến đầu ghi p[1] trên toàn hệ thống đã được đóng**.

Quy tắc EOF trên là điểm mấu chốt quyết định sự thành bại khi lập trình với pipe. Do tiến trình con sau fork() kế thừa toàn bộ các file descriptor của cha, nên cả hai tiến trình đều đồng thời giữ các bản sao của p[0] và p[1]. Nếu một tiến trình chỉ có nhiệm vụ đọc dữ liệu nhưng quên không đóng đầu ghi p[1] của chính nó, hệ thống vẫn ghi nhận còn ít nhất một đầu ghi đang mở. Do đó, khi bên kia ngừng gửi, thao tác read() sẽ không bao giờ nhận được giá trị $0$ mà bị treo vĩnh viễn trong trạng thái chờ (*deadlock*).

Trong bài pingpong, việc đóng ngay các đầu pipe không sử dụng giúp hai tiến trình chuyển giao quyền thực thi tuần tự và tin cậy. Trong bài primes—hiện thực thuật toán sàng số nguyên tố đồng thời nổi tiếng của Doug McIlroy—các tiến trình con được kết nối nối tiếp nhau thành một chuỗi pipeline dài. Mỗi tiến trình đọc số nguyên tố đầu tiên từ đường ống bên trái, in ra màn hình, rồi tạo một tiến trình con bên phải để chuyển tiếp các số không chia hết cho số nguyên tố đó. Việc đóng triệt để đầu ghi cũ trước khi đọc dữ liệu mới là điều kiện bắt buộc duy nhất để tín hiệu EOF có thể lan truyền xuyên suốt toàn bộ chuỗi pipeline, giúp các tiến trình tự động kết thúc tuần tự từ trái sang phải mà không làm cạn kiệt bảng file descriptor của hệ thống.

**Cơ chế nhịp đồng hồ và ngắt định thời của sleep()**

Khác với hàm sleep() trong các hệ điều hành thông thường vốn nhận tham số tính theo giây, system call sleep(ticks) trong xv6 hoạt động dựa trên nhịp ngắt của bộ định thời phần cứng (*timer interrupt*). Sau mỗi khoảng thời gian định kỳ, vi xử lý RISC-V kích hoạt một ngắt hẹn giờ, chuyển quyền điều khiển cho hàm xử lý ngắt trong kernel để tăng biến toàn cục ticks lên 1 đơn vị, đồng thời đánh thức các tiến trình đang chờ trên biến này.

Khi chương trình gọi sleep(n), kernel chuyển tiến trình sang trạng thái ngủ và nhường CPU cho các tiến trình khác. Sau khi đủ $n$ nhịp ngắt trôi qua, tiến trình mới được chuyển lại sang trạng thái sẵn sàng chạy (RUNNABLE) để bộ lập lịch cấp phát CPU. Cơ chế này loại bỏ hoàn toàn việc lãng phí chu kỳ xử lý do vòng lặp bận (*busy-waiting*), nhưng cũng đồng nghĩa với việc khoảng thời gian chờ thực tế phụ thuộc vào tần số ngắt của máy ảo QEMU và tải của hệ thống, không phản ánh trực tiếp thời gian thực tính bằng giây.

## 3.5 Hệ thống tập tin và kỹ thuật duyệt cây thư mục

Hệ thống tập tin của xv6 tổ chức dữ liệu theo mô hình phân cấp dạng cây kinh điển của Unix. Để thao tác chính xác với tập tin và thư mục trong các tiện ích cp, tree, du, và diff, việc nắm vững cấu trúc nhị phân của Inode và Directory Entry là nền tảng tối quan trọng.

### Phân biệt Inode và Tên đường dẫn (Pathname)

Trong hệ thống tập tin xv6, mỗi tập tin được đại diện trên đĩa bởi một cấu trúc dữ liệu gọi là ***Inode*** (*index node*). Inode lưu trữ toàn bộ thông tin siêu dữ liệu (*metadata*) của tệp bao gồm: loại tệp (type: tệp thường T_FILE, thư mục T_DIR, hay thiết bị T_DEVICE), số hiệu thiết bị lưu trữ (dev), số Inode (ino), số lượng liên kết cứng (nlink), kích thước tệp tính theo byte (size), và địa chỉ các khối dữ liệu trên đĩa.

Điểm cốt lõi là: **Inode không lưu tên của tệp tin**. Tên tệp chỉ tồn tại bên trong các thư mục dưới dạng các mục tham chiếu. Do cơ chế liên kết cứng (*hard link*), nhiều tên đường dẫn khác nhau ở các vị trí khác nhau trong cây thư mục hoàn toàn có thể cùng trỏ về một Inode duy nhất trên đĩa.

Kiến thức này trực tiếp giải quyết bài toán an toàn dữ liệu trong tiện ích cp: Khi sao chép từ tệp nguồn sang tệp đích, nếu người dùng vô tình chỉ định hai đường dẫn trỏ về cùng một tệp vật lý (chẳng hạn cp a.txt a.txt hoặc qua hai liên kết cứng khác nhau), việc mở tệp đích với cờ cắt cụt (O_TRUNC) sẽ lập tức xóa sạch nội dung của tệp nguồn trước khi kịp sao chép, dẫn đến mất dữ liệu không thể phục hồi. Do đó, thay vì so sánh hai chuỗi đường dẫn (vốn dễ bị đánh lừa bởi đường dẫn tương đối), chương trình phải dùng stat() để đọc thông tin của cả hai tệp và kiểm tra cặp định danh vật lý duy nhất:

$$\text{st_src.dev} == \text{st_dst.dev} \quad \text{và} \quad \text{st_src.ino} == \text{st_dst.ino}$$

### Cấu trúc thư mục và bản ghi struct dirent

Trong kiến trúc của xv6, một thư mục thực chất là một tệp tin đặc biệt mà nội dung bên trong là danh sách liên tiếp các bản ghi cấu trúc thư mục (struct dirent), được định nghĩa trong kernel/fs.h:

- ushort inum: Số hiệu Inode mà mục này trỏ tới (2 byte).
- char name[DIRSIZ]: Tên của tệp hoặc thư mục con, có độ dài cố định tối đa 14 ký tự (DIRSIZ = 14).

Khi một chương trình mở thư mục bằng open(path, O_RDONLY), nó có thể dùng chính system call chuẩn read() để đọc tuần tự từng khối dữ liệu có kích thước đúng bằng sizeof(struct dirent). Khi phân tích danh sách này:

- Nếu bản ghi có de.inum == 0, điều đó có nghĩa là mục này đang trống hoặc đã bị xóa, chương trình cần bỏ qua.
- Chuỗi de.name có độ dài tối đa 14 ký tự và không được đảm bảo luôn có ký tự kết thúc chuỗi '\0' nếu tên tệp chiếm đủ 14 ký tự. Vì vậy, trước khi gọi các hàm xử lý chuỗi chuẩn như strcmp() hay in ra màn hình, chương trình

bắt buộc phải sao chép tên sang một bộ đệm an toàn và bảo đảm chèn ký tự '\0' vào cuối chuỗi.

### Các chế độ mở tập tin (*Open Flags*)

Khi mở một tệp bằng system call open(path, flags), chế độ mở được truyền dưới dạng các cờ bit kết hợp với nhau bằng toán tử |. Các cờ được định nghĩa trong kernel/fcntl.h bao gồm:

| Cờ truy cập (*Flags*) | Ý nghĩa kỹ thuật và ngữ cảnh sử dụng trong xv6 |
|---|---|
| O_RDONLY | Mở tập tin ở chế độ chỉ đọc (dùng khi đọc tệp nguồn trong cp, diff, duyệt thư mục trong tree, du). |
| O_WRONLY | Mở tập tin ở chế độ chỉ ghi (dùng khi ghi dữ liệu ra tệp đích trong cp). |
| O_RDWR | Mở tập tin cho phép đồng thời cả hai thao tác đọc và ghi dữ liệu. |
| O_CREATE | Tự động tạo tập tin mới nếu đường dẫn chưa tồn tại; nếu tệp đã tồn tại, giữ nguyên nội dung cũ. |
| O_TRUNC | Cắt cụt độ dài tập tin về 0 byte ngay khi mở, xóa toàn bộ nội dung cũ (thường kết hợp với O_WRONLY \| O_CREATE trong cp). |

### Kỹ thuật duyệt cây thư mục đệ quy (Recursive Directory Traversal)

Các tiện ích như tree và du đòi hỏi khả năng duyệt toàn bộ cây phân cấp thư mục. Thuật toán duyệt đệ quy dựa trên mô hình duyệt đồ thị theo chiều sâu (DFS):

1. **Điều kiện dừng:** Khi đường dẫn hiện tại trỏ đến một tệp thông thường (st.type == T_FILE), chương trình xử lý thông tin tệp (cập nhật dung lượng hoặc in tên) rồi quay lui.
2. **Xử lý nút nhánh:** Khi đường dẫn trỏ đến một thư mục (st.type == T_DIR), chương trình mở thư mục và đọc tuần tự từng bản ghi struct dirent.
3. **Loại trừ liên kết tự thân và liên kết cha:** Điểm tối quan trọng trong việc duyệt cây thư mục là **bắt buộc phải bỏ qua hai mục đặc biệt là "." (thư mục hiện hành) và ".." (thư mục cha)**. Nếu không lọc hai mục này, hàm đệ quy sẽ lặp vô tận giữa thư mục cha và thư mục con cho đến khi làm tràn ngăn xếp bộ nhớ (*stack overflow*) hoặc chạm ngưỡng giới hạn mở tệp của hệ điều hành.
4. **Ghép đường dẫn an toàn:** Với mỗi mục con hợp lệ, chương trình tạo đường dẫn mới theo định dạng path/name. Cần bảo đảm bộ đệm chứa đường dẫn

đủ lớn (ít nhất 512 byte trong xv6) và kiểm tra giới hạn độ dài trước khi ghép chuỗi để ngăn chặn triệt để lỗi tràn bộ đệm (*buffer overflow*).

## 3.6 Xử lý dòng dữ liệu I/O và nguyên tắc quản trị tài nguyên an toàn

Để các chương trình vận hành ổn định trên các tập dữ liệu có quy mô lớn hoặc trong các tình huống ngoại lệ, nhóm đã áp dụng các nguyên tắc kỹ thuật chặt chẽ về quản trị I/O và vòng đời tài nguyên.

**Xử lý I/O theo khối đệm (Chunk-based Buffer I/O)**

Trong các tiện ích sao chép dữ liệu như cp, việc đọc hoặc ghi từng byte đơn lẻ thông qua system call sẽ làm gia tăng chi phí chuyển ngữ cảnh (trap overhead) lên mức không thể chấp nhận được. Ngược lại, việc đọc toàn bộ nội dung tệp vào một vùng nhớ lớn trong RAM lại bất khả thi đối với các tệp có dung lượng vượt quá kích thước bộ nhớ khả dụng của tiến trình.

Giải pháp chuẩn mực được áp dụng là **kỹ thuật truyền dữ liệu qua bộ đệm theo khối (*buffer streaming*)**. Chương trình duy trì một vùng đệm tĩnh có kích thước cố định (ví dụ: $512$ byte, tương ứng đúng bằng kích thước một khối đĩa (*block size*) của hệ thống tệp xv6). Trong vòng lặp sao chép:

- Gọi n = read(src_fd, buf, sizeof(buf)). Giá trị $n > 0$ cho biết số byte thực tế đọc được; $n = 0$ thông báo đã chạm điểm kết thúc tệp (EOF); và $n < 0$ báo hiệu lỗi đọc từ đĩa.
- Khi $n > 0$, chương trình ghi đúng $n$ byte vào tệp đích bằng write(dst_fd, buf, n). Cần kiểm tra kỹ giá trị trả về để bảo đảm toàn bộ dữ liệu đã được ghi trọn vẹn, phòng ngừa trường hợp đầy đĩa.

**Kỹ thuật phân tích dòng văn bản trong diff**

Tiện ích diff đòi hỏi phải so sánh hai tệp tin theo từng dòng văn bản độc lập. Do xv6 không có sẵn các hàm xử lý dòng mức cao như getline() trong POSIX, nhóm tự xây dựng cơ chế đệm dòng:

- Dữ liệu từ đĩa được đọc theo từng khối vào một bộ đệm vòng.
- Một hàm trợ giúp sẽ quét tìm ký tự xuống dòng ('\n'). Toàn bộ các ký tự từ vị trí hiện tại đến trước '\n' được gom thành một dòng dữ liệu hoàn chỉnh.
- Xử lý chặt chẽ các trường hợp biên (*edge cases*): dòng cuối cùng của tệp không kết thúc bằng ký tự '\n' trước khi gặp EOF; tệp rỗng hoàn toàn ($0$ byte); và trường hợp dòng văn bản có độ dài vượt quá kích thước bộ đệm

ban đầu (cần cấp phát động lại vùng nhớ bằng malloc() và di chuyển dữ liệu bằng memmove()).

**Nguyên tắc giải phóng tài nguyên và xử lý lỗi**

Môi trường thực thi của xv6 áp đặt các ràng buộc tài nguyên rất nghiêm ngặt: mỗi tiến trình chỉ được phép mở tối đa NOFILE = 16 tập tin mô tả đồng thời, và toàn bộ hệ thống chỉ có tối đa 64 tiến trình chạy cùng lúc. Do đó, việc quản trị tài nguyên phải tuân thủ tuyệt đối các nguyên tắc:

1. **Đóng tập tin mô tả ngay khi hoàn tất:** Mọi fd thu được từ open() hoặc pipe() đều phải được gọi close() tương ứng. Trong các hàm có nhiều nhánh thoát lỗi (*early exit*), việc dọn dẹp descriptor phải được thực hiện ở tất cả các nhánh.
2. **Đóng sớm các đầu pipe không dùng:** Trong các chương trình đa tiến trình (pingpong, primes), các đầu ghi và đọc không thuộc trách nhiệm của tiến trình hiện tại phải được đóng *ngay lập tức* sau lời gọi fork(), không giữ lại cho đến cuối chương trình.
3. **Thu hồi bộ nhớ động:** Mọi khối nhớ được cấp phát bằng malloc() phải có lệnh free() đối ứng sau khi dùng xong để ngăn ngừa rò rỉ bộ nhớ (*memory leak*).
4. **Thông báo lỗi qua stderr và mã thoát chuẩn:** Khi một thao tác gặp sự cố (sai cú pháp tham số, không mở được tệp, lỗi đọc/ghi), chương trình luôn in thông báo chi tiết ra tập tin mô tả số 2 (stderr) thông qua fprintf(2, ...) và thoát với mã trạng thái khác 0 (exit(1)), cho phép các tiến trình gọi hoặc shell nhận biết chính xác trạng thái thực thi.

# 4. Các chương trình đã thực hiện

Trong phần này, nhóm trình bày chi tiết thiết kế kỹ thuật, giải thuật và giải pháp hiện thực của toàn bộ 07 chương trình tiện ích người dùng được phát triển trong khuôn khổ bài thực hành, tương ứng với các tệp mã nguồn đặt tại thư mục xv6-labs-2024/user/. Mỗi chương trình được phân tích từ đặc tả yêu cầu, cách tổ chức mã nguồn, luồng xử lý chính đến các kỹ thuật xử lý edge case và bảo đảm an toàn tài nguyên hệ thống.

## 4.1 sleep

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Tạm dừng tiến trình hiện tại trong một số chu kỳ xung nhịp ngắt định thời (ticks) do người dùng chỉ định.
- **Cú pháp và tham số:** Nhận đúng một tham số bắt buộc theo cú pháp sleep ticks, trong đó ticks phải là chuỗi số nguyên không âm hợp lệ.
- **Mã thoát và xử lý lỗi:** Thoát với mã 0 khi hoàn thành; thoát với mã 1 và thông báo lỗi rõ ràng ra stderr (fd = 2) nếu thiếu, thừa đối số hoặc tham số chứa ký tự không hợp lệ.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được nhóm hiện thực trong tệp user/sleep.c. Toàn bộ luồng xử lý được tổ chức gọn gàng trong hàm main(int argc, char *argv[]) theo các bước tuần tự:

1. **Kiểm tra số lượng đối số:** Kiểm tra điều kiện argc != 2. Nếu người dùng không nhập tham số hoặc nhập nhiều hơn một tham số, chương trình in cú pháp chuẩn fprintf(2, "Usage: sleep ticks\n") và thoát ngay bằng exit(1).
2. **Xác thực định dạng số nguyên:** Hàm atoi() của thư viện người dùng trong xv6 không có cơ chế phát hiện lỗi cú pháp (nó tự động dừng khi gặp ký tự lạ mà không thông báo lỗi). Do đó, nhóm đã chủ động duyệt qua từng ký tự của chuỗi argv[1]. Nếu chuỗi rỗng (*s == '\0') hoặc chứa bất kỳ ký tự nào nằm ngoài khoảng '0' đến '9' (*s < '0' || *s > '9'), chương trình lập tức in thông báo lỗi fprintf(2, "sleep: invalid ticks '%s'\n", argv[1]) và thoát với exit(1).
3. **Tạm dừng và kết thúc:** Sau khi chuỗi được xác thực hợp lệ, chương trình gọi atoi(argv[1]) để nhận giá trị số nguyên và chuyển giao quyền kiểm soát cho kernel thông qua system call sleep(ticks). Tiến trình nhường CPU trong đúng khoảng thời gian yêu cầu, sau đó được đánh thức và thoát sạch sẽ bằng exit(0).

Mã giả dưới đây mô tả chính xác thuật toán xử lý trong user/sleep.c:

sleep

```
1  PROCEDURE Main(argc, argv)
2      IF argc != 2 THEN
3          PRINT_ERR "Usage: sleep ticks"
4          EXIT 1
5      END IF
6      s ← argv[1]
7      IF *s == 0 THEN
8          PRINT_ERR "sleep: invalid ticks", argv[1]
9          EXIT 1
```

```
    END IF
    FOR EACH character c in s DO
      IF c < '0' OR c > '9' THEN
        PRINT_ERR "sleep: invalid ticks", argv[1]
        EXIT 1
      END IF
    END FOR
    ticks ← atoi(argv[1])
    sleep(ticks)
    EXIT 0
END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Phòng ngừa tham số phi số và chuỗi rỗng:** Bắt trọn vẹn các edge case như sleep "", sleep 10s, sleep -5, sleep abc, ngăn chặn hoàn toàn việc gọi nhầm sleep() với giá trị không xác định.
- **Tối ưu hiệu năng CPU:** Sử dụng cơ chế ngủ của kernel giúp tiến trình chuyển sang trạng thái SLEEPING, tránh tuyệt đối việc lãng phí tài nguyên CPU do chạy vòng lặp bận (*busy-waiting*).

## 4.2 pingpong

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Minh họa giao tiếp liên tiến trình (IPC) hai chiều đồng bộ giữa tiến trình cha và tiến trình con bằng cách trao đổi một byte qua cặp đường ống (*pipes*).
- **Cú pháp và tham số:** Lệnh được thực thi trực tiếp bằng cú pháp pingpong, không nhận tham số dòng lệnh.
- **Định dạng xuất và mã thoát:** Tiến trình con in "<PID>: received ping\n", tiến trình cha in "<PID>: received pong\n". Thoát với mã 0 khi hoàn thành; thoát với mã 1 nếu bất kỳ thao tác tạo pipe, fork hoặc truyền nhận dữ liệu nào bị lỗi.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/pingpong.c. Do mỗi đường ống trong Unix chỉ truyền dữ liệu một chiều (*simplex*), nhóm đã thiết lập một cặp đường ống để hiện thực kênh giao tiếp hai chiều hoàn chỉnh:

1. **Khởi tạo hai đường ống riêng biệt:** Khởi tạo ống p[2] (cho chiều truyền từ cha sang con) và ống q[2] (cho chiều phản hồi từ con về cha) trước khi

tách tiến trình. Nếu bất kỳ lời gọi pipe() nào thất bại, chương trình in thông báo lỗi ra stderr và thoát với exit(1).

2. **Tách nhánh bằng fork():**
   - **Tiến trình con (pid == 0):** Lập tức đóng đầu ghi p[1] và đầu đọc q[0]. Con chờ nhận 1 byte từ cha thông qua read(p[0], &c, 1). Khi nhận thành công, con in định danh và thông điệp printf("%d: received ping\n", getpid()), sau đó gửi lại chính byte đó sang cha qua write(q[1], &c, 1). Cuối cùng, con đóng toàn bộ các đầu pipe còn lại (p[0] và q[1]) rồi thoát với exit(0).
   - **Tiến trình cha (pid > 0):** Lập tức đóng đầu đọc p[0] và đầu ghi q[1]. Cha bắt đầu cuộc hội thoại bằng cách gửi 1 byte 'x' vào p[1], kiểm tra dữ liệu gửi thành công rồi đóng ngay p[1]. Tiếp theo, cha gọi wait(0) để tạm dừng và chờ con xử lý xong. Khi con đã phản hồi và kết thúc, cha đọc byte nhận được từ q[0], in thông điệp printf("%d: received pong\n", getpid()), đóng q[0] và thoát với exit(0).

Trình tự trao đổi và đồng bộ hóa giữa hai tiến trình được mô tả qua mã giả dưới đây:

pingpong

```
1  PROCEDURE Main(argc, argv)
2      Create pipe p (parent to child) and pipe q (child to parent)
3      IF pipe creation fails THEN
4          PRINT_ERR "pingpong: pipe failed"; EXIT 1
5      END IF
6      pid ← fork()
7      IF pid < 0 THEN
8          PRINT_ERR "pingpong: fork failed"; EXIT 1
9      ELSE IF pid > 0 THEN // Tiến trình cha
10         Close p[0]; Close q[1]
11         c ← 'x'
12         Write 1 byte c to p[1]; Close p[1]
13         wait(0) // Chờ tiến trình con hoàn tất
14         Read 1 byte from q[0] into c; Close q[0]
15         PRINT getpid(), ": received pong"
16     ELSE // Tiến trình con
17         Close p[1]; Close q[0]
18         Read 1 byte from p[0] into c
19         PRINT getpid(), ": received ping"
20         Write 1 byte c to q[1]
21         Close p[0]; Close q[1]
22     END IF
23     EXIT 0
24 END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Đảm bảo thứ tự hiển thị tuần tự:** Nhờ cơ chế chặn (*blocking*) tự nhiên của read() ở con và lệnh wait(0) ở cha, tiến trình con luôn in thông báo ping trước khi tiến trình cha in thông báo pong, loại bỏ hoàn toàn tình trạng tranh chấp hiển thị (*race condition*) mà không phụ thuộc vào thứ tự lập lịch CPU.
- **Đóng sớm các đầu pipe không dùng:** Việc đóng các đầu pipe thừa ngay sau fork() giúp giải phóng các mục trong bảng ofile và triệt tiêu nguy cơ tắc nghẽn (*deadlock*).
- **Thu hồi tiến trình con sạch sẽ:** Lời gọi wait(0) ở cha bảo đảm tiến trình con được giải phóng hoàn toàn khỏi bảng quản lý tiến trình, không để lại tiến trình ma (*zombie process*).

## 4.3 primes

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Tìm và in toàn bộ các số nguyên tố trong khoảng từ 2 đến 280 bằng giải thuật Sàng Eratosthenes đồng thời (*Concurrent Prime Sieve*) theo mô hình CSP (Communicating Sequential Processes) của Doug McIlroy.
- **Cú pháp và tham số:** Thực thi trực tiếp bằng lệnh primes, không nhận tham số dòng lệnh; phạm vi sàng được cố định từ 2 đến 280 theo yêu cầu bài thực hành.
- **Định dạng xuất và mã thoát:** Mỗi số nguyên tố tìm được được in trên một dòng riêng biệt theo định dạng "prime <n>\n". Toàn bộ pipeline tiến trình phải kết thúc sạch sẽ với mã 0.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/primes.c. Thay vì dùng một mảng đánh dấu trong bộ nhớ như giải thuật tuần tự truyền thống, nhóm xây dựng một chuỗi đường ống dẫn dữ liệu gồm nhiều tiến trình chạy đồng thời, trong đó mỗi tiến trình đóng vai trò là một bộ lọc số nguyên tố độc lập:

1. **Tiến trình khởi tạo (main):** Tạo đường ống ban đầu pipe(p) rồi gọi fork():
   - Tiến trình cha đóng đầu đọc close(p[0]), đóng vai trò là nguồn cấp số (*number generator*), đẩy tuần tự các số nguyên từ 2 đến 280 vào p[1]. Sau khi ghi xong, cha đóng close(p[1]) để phát tín hiệu EOF và gọi wait(0) chờ toàn bộ chuỗi pipeline hoàn tất.

- Tiến trình con đóng đầu ghi close(p[1]) và gọi hàm đệ quy primes(p[0]) để bắt đầu tầng sàng đầu tiên.

2. **Hàm xử lý một tầng sàng (void primes(int left)):**
   - **Xác định số nguyên tố của tầng:** Đọc số đầu tiên từ pipe vào: read(left, &prime, sizeof(prime)). Nếu không đọc đủ byte (bên trái đã đóng truyền EOF), tiến trình đóng left và thoát ngay bằng exit(0) (đây là điều kiện dừng đệ quy). Số đầu tiên nhận được chắc chắn là số nguyên tố; tiến trình in ngay ra màn hình: printf("prime %d\n", prime).
   - **Lọc và chuyển tiếp dữ liệu:** Vòng lặp while (read(left, &n, sizeof(n)) == sizeof(n)) tiếp tục đọc các số kế tiếp:
     - Nếu n % prime == 0: Bỏ qua vì n là bội số của số nguyên tố hiện tại.
     - Nếu n không chia hết: Áp dụng kỹ thuật **khởi tạo lười** (*lazy initialization*) cho kênh truyền bên phải (right). Chỉ khi nhận được số đầu tiên không chia hết, tiến trình mới tạo ống pipe(p) và tách tiến trình con mới bằng fork():
       * Tiến trình con mới sinh đóng đầu ghi close(p[1]), đóng pipe thừa kế của tầng cha close(left), rồi gọi đệ quy primes(p[0]).
       * Tiến trình cha đóng đầu đọc close(p[0]) và lưu right = p[1].

       Sau đó, tiến trình ghi n sang tầng kế tiếp: write(right, &n, sizeof(n)).
   - **Đóng kênh và đồng bộ:** Khi đọc hết dữ liệu từ left, tiến trình đóng close(left). Nếu đã tạo tầng bên phải (right >= 0), tiến trình đóng tiếp close(right) để kích hoạt EOF cho tầng con và gọi wait(0) chờ con kết thúc trước khi gọi exit(0).

Mã giả của giải thuật sàng đồng thời được trình bày chi tiết dưới đây:

primes

```
1  PROCEDURE Primes(left)
2    IF read(left, &prime, sizeof(prime)) != sizeof(prime) THEN
3      Close left; EXIT 0
4    END IF
5    PRINT "prime ", prime
6    right ← -1
7    WHILE read(left, &n, sizeof(n)) == sizeof(n) DO
8      IF n % prime == 0 THEN
9        CONTINUE
10     END IF
11     IF right < 0 THEN // Khởi tạo lười cho tầng kế
12       pipe(p); pid ← fork()
13       IF pid == 0 THEN // Tầng con
14         Close p[1]; Close left
```

```
15              Primes(p[0])
16          END IF
17          Close p[0]; right ← p[1]
18       END IF
19       Write n to right
20    END WHILE
21    Close left
22    IF right >= 0 THEN
23       Close right; wait(0)
24    END IF
25    EXIT 0
26 END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Kiểm soát ngưỡng File Descriptor (NOFILE = 16):** Đây là rủi ro lớn nhất của bài toán sàng đa tiến trình. Nhóm giải quyết triệt để bằng cách đóng ngay các đầu pipe thừa kế (p[1] và left) trong tiến trình con *trước khi* gọi đệ quy tầng tiếp theo, đảm bảo mỗi tiến trình chỉ giữ đúng 1 đầu đọc và tối đa 1 đầu ghi.
- **Cơ chế kích hoạt EOF dây chuyền (*Cascade EOF*):** Khi tiến trình main ghi hết số và đóng p[1], tầng sàng đầu tiên nhận EOF → đóng right → tầng thứ hai nhận EOF → lan truyền lần lượt đến cuối chuỗi. Nhờ đó, tất cả các tiến trình tự động giải phóng tuần tự mà không bị treo bế tắc (*deadlock*).
- **Thu hồi tiến trình sạch sẽ:** Lời gọi wait(0) ở mỗi tầng cha bảo đảm thu dọn toàn bộ các tiến trình con theo chiều ngược từ ngọn về gốc, không để rò rỉ bất kỳ tiến trình zombie nào.

## 4.4 cp

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Sao chép toàn bộ nội dung dữ liệu từ tệp tin nguồn (src) sang tệp tin đích (dst).
- **Cú pháp và tham số:** Nhận đúng hai tham số bắt buộc theo cú pháp cp src dst.
- **Điều kiện tiên quyết:** Tệp nguồn phải tồn tại và là tệp thông thường (T_FILE); nếu tệp đích đã tồn tại thì tệp đích cũng phải là tệp thông thường và **không được trùng với tệp nguồn**.
- **Mã thoát và xử lý lỗi:** Thoát với mã $0$ khi thành công; thoát với mã $1$ và thông báo lỗi rõ ràng ra stderr nếu sai cú pháp, không mở được tệp, lỗi

đọc/ghi đĩa hoặc phát hiện tệp nguồn và đích trùng nhau.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/cp.c với luồng xử lý tuần tự chặt chẽ:

1. **Kiểm tra đối số dòng lệnh:** Kiểm tra argc != 3. Nếu sai số lượng đối số, in hướng dẫn fprintf(2, "Usage: cp src dst\n") và thoát với exit(1).
2. **Mở và kiểm tra tệp nguồn:** Mở tệp nguồn với cờ O_RDONLY. Lấy siêu dữ liệu bằng fstat(srcfd, &srcstat). Nếu srcstat.type != T_FILE, chương trình in lỗi "cp: source must be a regular file\n", đóng srcfd và thoát.
3. **Kiểm tra an toàn tệp đích trước khi ghi đè:** Nếu tệp đích đã tồn tại (stat(argv[2], &dststat) == 0):
   - **Phát hiện tệp trùng lặp:** So sánh cặp số hiệu thiết bị và Inode: srcstat.dev == dststat.dev && srcstat.ino == dststat.ino. Nếu trùng nhau, chương trình in lỗi "cp: source and destination are the same file\n", đóng srcfd và thoát. Việc kiểm tra này bắt buộc phải thực hiện *trước* khi mở tệp đích với cờ O_TRUNC, nhằm ngăn chặn tuyệt đối nguy cơ tệp nguồn bị xóa trắng nếu người dùng gõ lệnh dạng cp a a hoặc thông qua hai liên kết cứng trỏ cùng Inode.
   - **Kiểm tra loại tệp đích:** Xác nhận tệp đích phải là tệp thường (dststat.type == T_FILE).
4. **Mở và cắt cụt tệp đích:** Mở tệp đích với tổ hợp cờ O_CREATE | O_WRONLY | O_TRUNC. Nếu thất bại, đóng srcfd và báo lỗi.
5. **Truyền dữ liệu theo khối (*Buffer Streaming*):**
   - Khởi tạo bộ đệm tĩnh char buf[512], tương ứng đúng kích thước một khối đĩa (*block size*) của hệ thống tệp xv6.
   - Vòng lặp while ((n = read(srcfd, buf, sizeof(buf))) > 0) đọc dữ liệu theo từng khối và ghi đúng $n$ byte vào tệp đích: if (write(dstfd, buf, n) != n). Nếu số byte ghi thực tế không bằng $n$, chương trình báo lỗi ghi đĩa, đóng cả hai tệp và thoát với exit(1).
   - Sau vòng lặp, kiểm tra n < 0 để phát hiện lỗi đọc tệp nguồn.
6. **Đóng tệp và kết thúc:** Đóng cả hai tập tin mô tả close(srcfd), close(dstfd) và kết thúc thành công với exit(0).

Mã giả chi tiết của tiện ích cp được trình bày dưới đây:

cp

```
PROCEDURE Main(argc, argv)
    IF argc != 3 THEN
        PRINT_ERR "Usage: cp src dst"; EXIT 1
    END IF
    srcfd ← open(argv[1], O_RDONLY)
    IF srcfd < 0 OR fstat(srcfd, &srcstat) < 0 THEN
        PRINT_ERR "cannot open/stat source"; EXIT 1
    END IF
    IF srcstat.type != T_FILE THEN
        PRINT_ERR "source must be regular file"; Close srcfd; EXIT 1
    END IF
    IF stat(argv[2], &dststat) == 0 THEN
        IF srcstat.dev == dststat.dev AND srcstat.ino == dststat.ino THEN
            PRINT_ERR "same file"; Close srcfd; EXIT 1
        END IF
        IF dststat.type != T_FILE THEN
            PRINT_ERR "dst must be regular file"; Close srcfd; EXIT 1
        END IF
    END IF
    dstfd ← open(argv[2], O_CREATE | O_WRONLY | O_TRUNC)
    IF dstfd < 0 THEN
        Close srcfd; EXIT 1
    END IF
    WHILE (n ← read(srcfd, buf, sizeof(buf))) > 0 DO
        IF write(dstfd, buf, n) != n THEN
            Close srcfd; Close dstfd; EXIT 1
        END IF
    END WHILE
    IF n < 0 THEN
        Close srcfd; Close dstfd; EXIT 1
    END IF
    Close srcfd; Close dstfd
    EXIT 0
END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Phòng chống mất dữ liệu do tệp trùng nhau:** Bắt chính xác trường hợp tệp nguồn và tệp đích là một dựa trên cặp định danh (dev, ino), bảo đảm không bị xóa trắng nội dung tệp nguồn khi mở đích bằng O_TRUNC.
- **Xử lý tệp kích thước bất kỳ:** Nhờ cơ chế streaming theo khối 512 byte, chương trình xử lý mượt mà từ tệp rỗng (0 byte) đến các tệp có dung lượng lớn mà không làm tiêu tốn bộ nhớ RAM.
- **Quản lý đóng Descriptor triệt để:** Mọi điểm rẽ nhánh lỗi (*early exit*) đều được giải phóng toàn bộ các file descriptor đang mở, tránh rò rỉ tài

nguyên hệ thống.

## 4.5 tree

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Hiển thị cấu trúc cây phân cấp của thư mục và tệp tin một cách trực quan bằng các ký tự thụt lề nhánh "|__ ".
- **Cú pháp và tham số:** tree [path] [-d] [-L depth], trong đó:
  - path: Đường dẫn thư mục gốc cần duyệt (tùy chọn, mặc định là "." — thư mục hiện hành).
  - -d: Tùy chọn chỉ hiển thị thư mục (*directories only*), bỏ qua các tệp tin thường.
  - -L depth: Giới hạn độ sâu tối đa của cây duyệt (depth > 0).
- **Mã thoát và xử lý lỗi:** Thoát với mã 0 khi thành công; thoát với mã 1 và thông báo chi tiết ra stderr nếu tùy chọn không hợp lệ, thiếu đối số cho -L, giá trị độ sâu không dương, hoặc không thể truy cập đường dẫn gốc.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/tree.c, chia thành 3 phần rõ ràng:

1. **Phân tích đối số dòng lệnh (main):** Khởi tạo các biến cấu hình toàn cục: path = ".", max_depth = -1, only_dir = 0. Vòng lặp duyệt argv để bắt cờ -d, cờ -L (kèm kiểm tra thiếu tham số i + 1 >= argc và xác thực max_depth = atoi(argv[++i]) > 0), phát hiện cờ lạ argv[i][0] == '-', và nhận diện đường dẫn chỉ định.
2. **Hàm điều phối gốc (void tree(char *path)):** Kiểm tra đường dẫn gốc bằng stat(path, &st). Nếu là tệp thường, in tên tệp rồi dừng (nếu không bật -d). Nếu là thư mục, in tên thư mục gốc và gọi hàm duyệt đệ quy tree_traverse(path, 0).
3. **Hàm duyệt đệ quy (void tree_traverse(char *path, int level)):**
   - **Điều kiện dừng độ sâu:** Nếu max_depth != -1 && level >= max_depth, hàm lập tức quay lui (return).
   - **Mở thư mục và kiểm tra tràn đệm:** Mở thư mục fd = open(path, O_RDONLY). Kiểm tra điều kiện an toàn độ dài: strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf) (với buf[256]). Nếu vượt quá ngưỡng an toàn, in thông báo lỗi "tree: path too long\n", đóng fd và quay lui.
   - **Ghép tiền tố đường dẫn:** Sao chép path vào buf và chèn dấu '/' ở cuối.

- **Đọc và lọc các bản ghi struct dirent:** Vòng lặp while (read(fd, &de, sizeof(de)) == sizeof(de)):
  - Bỏ qua entry trống: de.inum == 0.
  - **Bỏ qua mục đặc biệt:** !strcmp(de.name, ".") || !strcmp(de.name, "..").
  - Chuẩn hóa chuỗi tên tệp kết thúc bằng NULL: name[DIRSIZ] = 0.
  - Nối tên vào đường dẫn đầy đủ buf và lấy thông tin stat(buf, &st). Nếu tệp không đọc được thông tin, in cảnh báo và bỏ qua.
  - Nếu cờ only_dir bật mà st.type != T_DIR, bỏ qua không in.
  - In nhánh thụt lề bằng hàm trợ giúp print_indent(level + 1) (in các khoảng trắng " " và ký tự "|__ ") kèm tên name.
  - Nếu mục là thư mục con (st.type == T_DIR), gọi đệ quy tiếp tục: tree_traverse(buf, level + 1).
- Đóng thư mục close(fd) ngay khi duyệt hết các entry của tầng hiện tại.

Mã giả của giải thuật duyệt cây thư mục được mô tả chi tiết dưới đây:

tree

```
PROCEDURE TreeTraverse(path, level)
    IF max_depth != -1 AND level >= max_depth THEN
        RETURN
    END IF
    fd ← open(path, O_RDONLY)
    IF fd < 0 THEN
        PRINT_ERR "cannot open ", path; RETURN
    END IF
    IF strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf) THEN
        PRINT_ERR "path too long"; Close fd; RETURN
    END IF
    Format buf with path and trailing '/'
    WHILE read(fd, &de, sizeof(de)) == sizeof(de) DO
        IF de.inum == 0 OR de.name is "." OR ".." THEN
            CONTINUE
        END IF
        Construct child path in buf; stat(buf, &st)
        IF only_dir AND st.type != T_DIR THEN
            CONTINUE
        END IF
        PrintIndent(level + 1); PRINT de.name
        IF st.type == T_DIR THEN
            TreeTraverse(buf, level + 1)
        END IF
    END WHILE
    Close fd
END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Triệt tiêu lặp vô hạn:** Bắt buộc bỏ qua hai mục "." và ".." giúp ngăn chặn hàm đệ quy chạy quẩn giữa thư mục con và cha, bảo vệ hệ thống khỏi lỗi tràn ngăn xếp bộ nhớ (*stack overflow*).
- **Phòng chống tràn đệm đường dẫn:** Cơ chế kiểm tra độ dài đường dẫn trước khi ghép chuỗi bảo đảm an toàn cho bộ đệm cố định 256 byte.
- **Đóng sớm Descriptor ở mỗi tầng đệ quy:** Việc đóng close(fd) ngay khi quét xong thư mục bảo đảm số lượng descriptor đồng thời chỉ tỷ lệ thuận với độ sâu tối đa của cây duyệt, không vượt quá giới hạn NOFILE = 16.

## 4.6 du

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** Ước tính dung lượng sử dụng đĩa (*Disk Usage*) của thư mục hoặc tệp tin theo đơn vị byte.
- **Cú pháp và tham số:** du [-a] [-s] [path], trong đó:
  - path: Đường dẫn cần tính dung lượng (mặc định là "." — thư mục hiện hành).
  - -a: Hiển thị chi tiết dung lượng của từng tệp tin bên cạnh các thư mục (*all files*).
  - -s: Chế độ tóm tắt (*summary*), chỉ hiển thị một dòng duy nhất ghi tổng dung lượng của đường dẫn gốc.
- **Định dạng xuất và mã thoát:** Hiển thị theo định dạng "<dung_lượng>\t<đường_dẫn> Thoát với mã 0 khi thành công; thoát với mã 1 và thông báo lỗi nếu gặp tùy chọn không hợp lệ hoặc không truy cập được đường dẫn gốc.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/du.c. Giải thuật tính dung lượng sử dụng mô hình duyệt hậu thứ tự (*post-order traversal*), bảo đảm các mục con được tính toán và tích lũy trước khi hoàn tất thư mục cha:

1. **Phân tích đối số dòng lệnh (main):** Khởi tạo các cờ cấu hình: all_files = 0, summary = 0, path = ".". Vòng lặp duyệt argv để nhận diện cờ -a, -s, báo lỗi và thoát khi gặp cờ không xác định (argv[i][0] == '-'), và nhận diện đường dẫn chỉ định. Sau đó, gọi stat(path, &st) để xác thực đường dẫn gốc trước khi kích hoạt hàm đệ quy total = du(path).
2. **Hàm tính dung lượng đệ quy (uint64 du(char *path)):**

- **Xác định loại mục:** Dùng stat(path, &st). Nếu lỗi, in thông báo ra stderr và trả về 0.
- **Trường hợp tệp tin thường hoặc thiết bị (T_FILE || T_DEVICE):** Nếu cờ all_files bật và không ở chế độ summary, chương trình in kích thước tệp printf("%d\t%s\n", (int)st.size, path). Hàm trả về kích thước st.size (tính theo byte).
- **Trường hợp thư mục (T_DIR):** Khởi tạo biến tích lũy uint64 total = 0. Mở thư mục bằng fd = open(path, O_RDONLY). Kiểm tra giới hạn độ dài đường dẫn vào bộ đệm tĩnh: strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf) (với buf[256]). Chuẩn bị chuỗi tiền tố với dấu '/'.
- **Đọc và tích lũy từ các mục con:** Vòng lặp while (read(fd, &de, sizeof(de)) == sizeof(de)):
  - Bỏ qua entry chưa dùng: de.inum == 0.
  - **Loại trừ liên kết tự thân và liên kết cha:** strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0.
  - Ghép tên mục con vào đường dẫn đầy đủ buf.
  - Gọi đệ quy uint64 child_size = du(buf) và cộng dồn: total += child_size.
- Đóng thư mục close(fd).
- **In kết quả thư mục:** Nếu không bật cờ summary, in dung lượng tổng của thư mục hiện tại: printf("%d\t%s\n", (int)total, path).
- Trả về giá trị total.

3. **Xử lý in kết quả cấp cao (main):** Nếu người dùng bật chế độ tóm tắt (summary), hàm main in dòng duy nhất printf("%d\t%s\n", (int)total, path). Nếu đầu vào là một tệp đơn lẻ và không bật -a, main cũng in dòng kết quả của tệp đó.

Mã giả của giải thuật tính dung lượng du được trình bày chi tiết dưới đây:

du

```
PROCEDURE Du(path)
    IF stat(path, &st) < 0 THEN
        PRINT_ERR "cannot stat ", path; RETURN 0
    END IF
    IF st.type == T_FILE OR st.type == T_DEVICE THEN
        IF all_files AND NOT summary THEN
            PRINT st.size, "\ t", path
        END IF
        RETURN st.size
    END IF
    IF st.type == T_DIR THEN
```

```
12        fd ← open(path, O_RDONLY)
13        IF fd < 0 OR strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf) THEN
14           Close fd if opened; RETURN 0
15        END IF
16        total ← 0; Format buf with path and '/'
17        WHILE read(fd, &de, sizeof(de)) == sizeof(de) DO
18           IF de.inum == 0 OR de.name is "." OR ".." THEN
19              CONTINUE
20           END IF
21           Construct child path in buf
22           total ← total + Du(buf)
23        END WHILE
24        Close fd
25        IF NOT summary THEN
26           PRINT total, "\ t", path
27        END IF
28        RETURN total
29     END IF
30     RETURN 0
31  END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Duyệt hậu thứ tự chính xác:** Bảo đảm dung lượng của toàn bộ các tệp và thư mục con bên trong được tính toán đầy đủ trước khi in tổng dung lượng của thư mục cha.
- **Phòng chống tràn số:** Khai báo biến tích lũy với kiểu dữ liệu số nguyên không dấu 64-bit (uint64), ngăn ngừa hiện tượng tràn số khi tổng kích thước tệp vượt quá giới hạn 32-bit.
- **Giải phóng tài nguyên sớm:** Thư mục được đóng close(fd) ngay khi đọc xong các bản ghi, không giữ descriptor trong lúc in kết quả hoặc trả về, tránh cạn kiệt bảng ofile.

## 4.7 diff

**Mục tiêu và đặc tả yêu cầu.**

- **Chức năng:** So sánh nội dung của hai tệp tin theo từng dòng (*line-by-line comparison*) và xác định các vị trí xuất hiện sự sai khác.
- **Cú pháp và tham số:** Hỗ trợ hai dạng cú pháp linh hoạt: diff file1 file2 [-q] hoặc diff -q file1 file2, trong đó:
  - file1, file2: Hai đường dẫn tệp tin thường cần so sánh nội dung. Chương

trình yêu cầu tên tệp không bắt đầu bằng ký tự '-' (nếu có phải dùng tiền tố ./-file) để tránh nhầm lẫn với cờ tùy chọn.

  - -q: Chế độ im lặng (*quiet mode*), chỉ thông báo hai tệp có khác nhau hay không mà không in chi tiết nội dung từng dòng sai lệch.

- **Quy ước mã thoát (*Exit Status*):** Tuân thủ nghiêm ngặt chuẩn của bài thực hành:
  - Mã 0: Hai tệp có nội dung hoàn toàn giống nhau (*identical*).
  - Mã 1: Hai tệp có sự khác biệt về nội dung (*different*).
  - Mã 2: Lỗi hệ thống hoặc lỗi thực thi (*error*), bao gồm sai cú pháp dòng lệnh, không mở được tệp, tệp không phải dạng tệp thông thường (T_FILE), lỗi I/O khi đọc/ghi hoặc cạn kiệt bộ nhớ heap.
- **Định dạng xuất kết quả:**
  - Ở chế độ thông thường, với mỗi vị trí khác biệt tại dòng thứ $N$, chương trình in hai dòng đối chiếu:

    <file1>:<N>: < <nội_dung_1>
    <file2>:<N>: > <nội_dung_2>

    Nếu một tệp kết thúc trước tệp còn lại, dòng tương ứng của tệp đó được gán nhãn EOF. Nếu dòng cuối cùng của tệp không kết thúc bằng ký tự xuống dòng (\n), chương trình xuất cảnh báo chuẩn \ No newline at end of file.
  - Ở chế độ -q, khi phát hiện điểm khác biệt đầu tiên, chương trình chỉ in duy nhất thông báo "diff: files differ\n" rồi lập tức kết thúc.

**Thiết kế kỹ thuật và cài đặt chi tiết.**

Chương trình được hiện thực tại user/diff.c. Điểm thách thức lớn nhất trong môi trường xv6 là hệ điều hành không cung cấp sẵn thư viện đệm dòng tiêu chuẩn như getline() hay fgets() trong libc của Linux. Nhóm đã tự thiết kế cấu trúc quản lý dòng động và cơ chế đọc/so sánh nhị phân an toàn:

1. **Cấu trúc dòng động (struct line):** Bao gồm bốn trường quản lý:
   - char *data: Con trỏ trỏ đến vùng nhớ đệm chứa chuỗi ký tự trên heap.
   - int len: Độ dài thực tế tính theo byte của dòng. Việc lưu trữ len tường minh cho phép chương trình xử lý an toàn cả các tệp nhị phân có chứa byte '\0' mà không bị ngắt chuỗi sớm.
   - int capacity: Dung lượng tối đa hiện tại của vùng đệm đã cấp phát.
   - int has_newline: Cờ đánh dấu dòng có ký tự kết thúc '\n' hay kết thúc đột ngột tại cuối tệp (EOF).

2. **Hàm đọc dòng tự mở rộng bộ đệm (readline):**
   - Khởi tạo bộ đệm ban đầu với kích thước 128 byte thông qua malloc(128).
   - Đọc tuần tự từng byte một từ tệp bằng read(fd, &c, 1).
   - Khi số byte tích lũy đạt ngưỡng giới hạn (line->len + 1 >= line->capacity), chương trình thực hiện nhân đôi dung lượng đệm (capacity = line->capacity * 2), cấp phát vùng nhớ mới, sao chép dữ liệu cũ qua memmove, giải phóng bộ nhớ cũ free(line->data) và cập nhật con trỏ. Kỹ thuật này bảo đảm chương trình đọc được những dòng có độ dài tùy ý mà không làm tràn bộ đệm hay giới hạn độ dài dòng cố định.
   - Khi gặp ký tự '\n', gán has_newline = 1 và trả về 1. Nếu gặp EOF khi chưa đọc được byte nào, trả về 0. Nếu gặp EOF khi đã có dữ liệu (dòng cuối thiếu ký tự xuống dòng), trả về 1 nhưng giữ has_newline = 0.
3. **Hàm xuất dòng an toàn (printline):** In tiền tố "%s:%d: %s " chứa đường dẫn, số thứ tự dòng và ký hiệu đánh dấu (< hoặc >). Nếu tệp đã hết dữ liệu (!present), in chuỗi "EOF\n". Ngược lại, hàm dùng vòng lặp gọi trực tiếp write(1, line->data + written, ...) dựa trên trường len thay vì dùng printf("%s") để bảo toàn trọn vẹn mọi byte dữ liệu (kể cả byte null). Sau đó, in ký tự xuống dòng và in thêm cảnh báo "\\ No newline at end of file\n" nếu cờ has_newline không được bật.
4. **Vòng lặp đối chiếu và điều phối (main):**
   - **Xác thực tệp tin:** Sau khi mở hai tệp (open), chương trình sử dụng fstat để kiểm tra loại tệp. Nếu một trong hai không phải là tệp thường (st.type != T_FILE), chương trình báo lỗi "diff: inputs must be regular files\n" và thoát với mã 2.
   - **So sánh song song từng cặp dòng:** Duyệt biến đếm dòng number từ 1. Tại mỗi bước, gọi đồng thời readline(fd1, &line1) và readline(fd2, &line2). Vòng lặp dừng khi cả hai tệp cùng chạm EOF (has1 == 0 && has2 == 0).
   - **Điều kiện xác định sai khác:** Hai dòng được coi là khác nhau nếu thỏa mãn một trong các điều kiện:
     - Một tệp còn dòng trong khi tệp kia đã hết (has1 != has2).
     - Độ dài byte của hai dòng không bằng nhau (line1.len != line2.len).
     - Trạng thái ký tự kết thúc dòng khác nhau (line1.has_newline != line2.has_newlin
     - So sánh nội dung byte qua memcmp(line1.data, line2.data, line1.len) != 0.
   - Khi phát hiện khác biệt, gán status = 1. Nếu ở chế độ quiet, in thông báo "diff: files differ\n" và ngắt vòng lặp. Nếu ở chế độ thường, gọi printline

để in chi tiết cả hai phía.

- **Thu dọn tài nguyên tại nhãn done:** Giải phóng toàn bộ vùng đệm heap (free(line1.data), free(line2.data)), đóng các file descriptor (close(fd1), close(fd2)) và kết thúc với exit(status).

Mã giả dưới đây khái quát logic so sánh hai tệp tin của chương trình diff:

diff

```
1  PROCEDURE Main(arguments)
2      Parse file paths path1, path2 and optional flag -q
3      IF syntax invalid OR path starts with '-' THEN
4          PRINT_ERR usage message; EXIT 2
5      END IF
6      Open fd1 and fd2; Verify both are T_FILE using fstat
7      IF open or stat fails THEN
8          PRINT_ERR error message; Clean up; EXIT 2
9      END IF
10     status ← 0; number ← 1; Initialize line1, line2
11     LOOP
12         has1 ← ReadLine(fd1, line1)
13         has2 ← ReadLine(fd2, line2)
14         IF has1 < 0 OR has2 < 0 THEN
15             PRINT_ERR "diff: read error or out of memory"
16             status ← 2; BREAK
17         END IF
18         IF has1 == 0 AND has2 == 0 THEN
19             BREAK
20         END IF
21         differs ← (has1 != has2) OR (line1.len != line2.len) OR
22                 (line1.has_newline != line2.has_newline) OR
23                 (memcmp(line1.data, line2.data, line1.len) != 0)
24         IF differs THEN
25             status ← 1
26             IF quiet THEN
27                 PRINT "diff: files differ"
28                 BREAK
29             END IF
30             PrintLine(path1, number, "<", has1, line1)
31             PrintLine(path2, number, ">", has2, line2)
32         END IF
33         number ← number + 1
34     END LOOP
35     Free line1.data and line2.data; Close fd1 and fd2
36     EXIT status
37 END PROCEDURE
```

**Xử lý edge case và an toàn tài nguyên.**

- **Xử lý tệp rỗng và tệp có độ dài lệch nhau:** Khi một tệp rỗng hoặc có số dòng ít hơn, biến has tương ứng trả về 0. Hàm printline nhận diện !present và in nhãn EOF, bảo đảm đối chiếu trực quan chính xác giữa phần dữ liệu thừa của tệp dài hơn và trạng thái kết thúc của tệp ngắn hơn.
- **Phân biệt ký tự xuống dòng ở cuối tệp:** Nhờ cờ has_newline, chương trình phân biệt chuẩn xác trường hợp hai dòng có cùng chuỗi ký tự nhưng một bên có '\n' còn bên kia kết thúc ngay tại EOF, đồng thời in cảnh báo chuẩn POSIX "\ No newline at end of file".
- **An toàn với dữ liệu nhị phân:** Bằng cách lưu trường len tường minh, dùng memcmp để so khớp nội dung và dùng trực tiếp lời gọi hệ thống write(1, data, len) thay vì printf("%s"), giải thuật không bị ngắt quãng bởi các byte null ('\0') lồng bên trong tệp.
- **Quản lý bộ nhớ và bộ mô tả tệp chặt chẽ:** Cấu trúc nhãn thu dọn done: bảo đảm vùng nhớ cấp phát động trên heap (malloc) luôn được giải phóng và các file descriptor luôn được đóng trước khi tiến trình kết thúc, ngăn ngừa rò rỉ tài nguyên hệ thống kể cả khi gặp lỗi bất thường giữa chừng.

# 5. Kết quả kiểm thử tổng hợp

## 5.1 Môi trường và cách thực hiện

Kiểm thử được thực hiện tự động trên xv6 chạy bằng QEMU 8.2.2, với 128 MB bộ nhớ và 3 CPU mô phỏng. Mã nguồn được biên dịch trên Ubuntu 24.04 trong WSL bằng GCC 13.3.0 cho RISC-V. Bộ chạy kiểm thử tạo một bản sao tạm của xv6, biên dịch lại chương trình và tạo ảnh hệ thống tệp riêng; vì vậy, các tệp thử nghiệm không làm thay đổi fs.img trong thư mục làm việc.

Bộ kiểm thử gồm cptest, difftest, linetest và coretest, đặt tại tests/lab01/. Các bài kiểm tra đối chiếu kết quả với giá trị mong đợi, bao gồm nội dung tệp, thông báo xuất ra và mã thoát. Riêng sleep, thời gian chờ được đo bằng bộ đếm tick của xv6. Lệnh chạy từ thư mục gốc của dự án, trong Ubuntu WSL, là:

```
python3 tests/lab01/run.py \
  --log docs/test-results/lab01-qemu.log
```

## 5.2 Kết quả theo từng chương trình

Lần chạy ghi nhận **77/77 phép kiểm tra đạt**, không xuất hiện thông báo FAIL và bộ chạy kiểm thử kết thúc với mã 0. Con số này bao gồm các lần chạy lặp lại và một kiểm tra tạo liên kết cứng phục vụ thử nghiệm cp; không phải số lượng trường hợp độc lập hay tỷ lệ bao phủ mã nguồn.

| Đối tượng | Phạm vi kiểm tra | Kết quả ghi nhận | Đạt/tổng |
|---|---|---|---|
| sleep | Chờ 5 tick, 0 tick; thiếu, thừa, sai định dạng và số âm | Lượt sleep 5 đo được 5 tick; thông báo và mã thoát đúng kỳ vọng | 6/6 |
| pingpong | Chạy 3 lần; kiểm tra thứ tự thông báo và PID | Mỗi lần in ping trước pong; hai PID khác nhau; mã thoát 0 | 3/3 |
| primes | Chạy 2 lần; đối chiếu dãy bằng phép thử chia độc lập | Đủ 59 số nguyên tố từ 2 đến 280, đúng thứ tự, không thừa hoặc trùng | 2/2 |
| cp | Tệp 0–4097 byte; ghi đè; cùng tệp, liên kết cứng; đối số và đường dẫn lỗi | Đúng từng byte và kích thước; giữ nguyên nguồn trong các ca từ chối sao chép | 18/18 |
| tree | Cây nhiều tầng; -d, -L 1, -L 2; tệp, thư mục rỗng và đầu vào lỗi | Cấu trúc, thụt lề, giới hạn độ sâu, thông báo và mã thoát đúng kỳ vọng | 11/11 |
| du | Tổng thư mục; -a, -s, kết hợp hai cờ; tệp đơn và đầu vào lỗi | Tệp mẫu 3, 5, 2 byte cho tổng 10 byte; kết quả chi tiết và tóm tắt đúng | 8/8 |
| diff | Tệp rỗng; dòng khác nhau; -q; hết tệp, dấu xuống dòng; dòng dài và byte 0 | Nội dung xuất và mã thoát 0/1/2 đúng kỳ vọng, kể cả dòng 1200 byte | 20/20 |
| Đọc dòng | Dòng trống, dòng cuối, hết tệp, mở rộng bộ đệm, byte 0 và lỗi đọc | Phân biệt dòng trống với hết tệp; giữ đủ nội dung và nhận biết lỗi đọc | 9/9 |
| **Tổng** | | **Tất cả phép kiểm tra đều đạt** | **77/77** |

### 5.3 Nhật ký thực thi và phạm vi kết luận

Nhật ký QEMU đầy đủ được lưu tại docs/test-results/lab01-qemu.log. Trích các dòng xác nhận hoàn tất từ lần chạy thực tế:

```
ALL CP TESTS PASSED
ALL DIFF TESTS PASSED
ALL LINE TESTS PASSED
MEASURE: sleep 5 elapsed=5 ticks
ALL SLEEP TESTS PASSED
ALL PINGPONG TESTS PASSED
ALL PRIMES TESTS PASSED
ALL TREE TESTS PASSED
ALL DU TESTS PASSED
ALL CORE TESTS PASSED
```

Các kết quả trên xác nhận hành vi của chương trình trong những trường hợp đã kiểm tra, không chứng minh chương trình không còn lỗi. Bộ test chưa mô phỏng đầy đĩa, thiếu bộ nhớ, cạn tài nguyên tiến trình hoặc mọi giới hạn đường dẫn; mẫu sao chép lớn nhất là 4097 byte và mẫu dòng dài là 1200 byte. Thời gian 5 tick là số đo của lượt chạy này, không phải cam kết tiến trình luôn được lập lịch lại đúng thời điểm trên mọi lần chạy.

Đây là **bộ kiểm thử riêng cho phạm vi Lab 01 của báo cáo**, không phải kết quả make grade. Bộ chấm grade-lab-util có trong kho còn kiểm tra find và xargs, nhưng không bao gồm cp, tree, du, diff. Vì chưa chạy toàn bộ bộ chấm đó trong lần kiểm thử này, báo cáo không ghi điểm chấm MIT hoặc quy đổi 77/77 thành điểm trên 100.

## 6. Khó khăn gặp phải và giải pháp khắc phục

1. **Vấn đề 1: Tràn số lượng File Descriptor trong bài primes.**

   *Nguyên nhân:* Tiến trình con kế thừa toàn bộ bảng file descriptor của tiến trình cha nhưng không đóng kịp thời các đầu ghi của các stage trước đó, dẫn đến cạn kiệt tài nguyên (vượt quá NOFILE trong kernel).

   *Giải pháp:* Đóng ngay lập tức các đầu pipe không dùng ở cả cha và con trước khi bước vào vòng lặp xử lý dữ liệu.

2. **Vấn đề 2: Lặp vô hạn khi duyệt thư mục trong tree và du.**

   *Nguyên nhân:* Bỏ sót việc lọc hai liên kết đặc biệt là thư mục hiện hành (.)

và thư mục cha (..).

*Giải pháp:* Bổ sung kiểm tra strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0 trước khi gọi hàm đệ quy.

# 7. Kết luận và hướng phát triển

Qua bài thực hành Lab 01, nhóm đã hoàn thiện toàn diện các kỹ năng lập trình không gian người dùng trên hệ điều hành xv6:

- Nắm vững cơ chế vận hành của system call và sự chuyển giao giữa User Space và Kernel Space.
- Làm chủ kỹ thuật điều khiển luồng tiến trình và đồng bộ hóa thông qua các cơ chế IPC cơ bản nhưng hiệu quả.
- Hiểu rõ cấu trúc tổ chức và lưu trữ dữ liệu của hệ thống tệp tin xv6 ở mức độ nhị phân.

# Tài liệu tham khảo

[1] Russ Cox, Frans Kaashoek, Robert Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, ngày 31/08/2024. Tham khảo chương 1 về system call và pipe, chương 8 về hệ thống tệp. https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf.

[2] MIT, học phần 6.1810, *Lab: Xv6 and Unix utilities*, Fall 2024. Yêu cầu và hướng dẫn cho sleep, pingpong, primes. https://pdos.csail.mit.edu/6.1810/2024/labs/util.html.

[3] Tài liệu hướng dẫn thực hành Hệ điều hành Lab 01 do Giảng viên bộ môn biên soạn.

[4] MIT PDOS, *xv6-riscv*, kho mã nguồn chính thức. Tham khảo cách tổ chức tiến trình, pipe, hệ thống tệp và chương trình ls. Mã nguồn thực hiện bài được đối chiếu trực tiếp trong thư mục xv6-labs-2024/ của dự án. https://github.com/mit-pdos/xv6-riscv.

[5] Russ Cox, *Bell Labs and CSP Threads*. Tham khảo mô hình sàng số nguyên tố bằng các tiến trình nối với nhau qua pipe. https://swtch.com/~rsc/thread/.

[6] MIT, học phần 6.1810, *Tools*, Fall 2024. Hướng dẫn chuẩn bị công cụ RISC-V, QEMU và môi trường chạy xv6.
https://pdos.csail.mit.edu/6.1810/2024/tools.html.

[7] GNU Project, *GNU Coreutils Manual*, các mục *cp invocation* và *du invocation*. Tham khảo chức năng và cách sử dụng các tiện ích sao chép, thống kê dung lượng trên Unix.
https://www.gnu.org/software/coreutils/manual/coreutils.html.

[8] GNU Project, *Comparing and Merging Files*, mục *Invoking diff*. Tham khảo cách diễn giải kết quả so sánh và mã thoát của diff.
https://www.gnu.org/software/diffutils/manual/html_node/Invoking-diff.html.

[9] Microsoft Learn, *How to install Linux on Windows with WSL*. Tham khảo yêu cầu hệ thống, cài bản phân phối Ubuntu và kiểm tra phiên bản WSL.
https://learn.microsoft.com/en-us/windows/wsl/install.

[10] Homebrew, *Installation*. Tham khảo cách cài đặt và yêu cầu môi trường macOS.
https://docs.brew.sh/Installation.

[11] Homebrew Formulae, *riscv64-elf-gcc* và *qemu*. Tham khảo tên gói, phụ thuộc Binutils và công cụ biên dịch RISC-V trên macOS.
https://formulae.brew.sh/formula/riscv64-elf-gcc;
https://formulae.brew.sh/formula/qemu.